

경구약제 이미지 객체 검출 프로젝트

Health Eat — 알약 인식 AI 모델 개발 및 성능 최적화 보고서

항목	내용
프로젝트명	경구약제 이미지 객체 검출 (Object Detection)
평가 지표	mAP@[0.75:0.95]
최종 캐글 점수	0.97581 (Public Leaderboard)
데이터셋	AI Hub 경구약제 이미지 — train 232장 / test 842장
클래스 수	56개 (test 40개)
개발 환경	Google Colab Pro (T4 GPU), PyTorch, Ultralytics
GitHub	twentycherry-gif/medicine-object-detection-project
제출자	최혜리
제출일	2026년 5월 10일

목차

- 프로젝트 개요
- 1.1 배경 및 목표 | 1.2 평가 방식 | 1.3 개발 환경
- 탐색적 데이터 분석 (EDA)
- 2.1 데이터셋 현황 | 2.2 촬영 환경 | 2.3 클래스 불균형
- 2.4 메타데이터 분석 | 2.5 주요 시각화 | 2.6 전처리 전략
- 모델 실험 과정
- 3.1 Round 1 — 아키텍처 탐색 | 3.2 Round 2 — 경량화 검토
- 3.3 Round 3 — YOLO 계열 세부 실험 | 3.4 모델별 성능 비교
- 양상불 전략
- 4.1 WBF vs Soft-NMS | 4.2 파라미터 튜닝 | 4.3 점수 변화 흐름
- 최종 결과
- 5.1 캐글 점수 히스토리 | 5.2 최적 조합 확정
- 주요 의사결정 근거

7. 한계점 및 개선 방향

8. 결론

1. 프로젝트 개요

1.1 배경 및 목표

헬스케어 스타트업 Health Eat의 AI 엔지니어로서, 사용자가 모바일 앱으로 복용 중인 알약 사진을 찍으면 해당 약에 대한 정보(약 이름, 성분, 제조사 등)를 제공하는 서비스를 구현합니다. 사진 속 최대 4개의 알약의 클래스(약 이름)와 위치(바운딩 박스)를 동시에 검출하는 것이 핵심 목표입니다.

본 프로젝트는 코드잇 스프린트 AI 엔지니어링 10기 초급 프로젝트로, Kaggle Private Competition 방식으로 성능을 평가합니다.

1.2 평가 방식

평가 지표는 mAP@[0.75:0.95]로, IoU 0.75~0.95 구간에서의 평균 정밀도(mean Average Precision)를 측정합니다. 단순한 객체 검출이 아니라 바운딩 박스의 정확한 위치 추정이 필수적이며, 하루 최대 10회 제출이 가능합니다.

1.3 개발 환경

항목	사양
플랫폼	Google Colab Pro
GPU	Tesla T4 (14GB VRAM)
스토리지	Google Drive (드라이브 마운트)
프레임워크	PyTorch 2.x, Ultralytics 8.4.46
주요 라이브러리	OpenCV, NumPy, Pandas, Pillow, Albumentations, ensemble-boxes
버전 관리	GitHub (twentycherry-gif/medicine-object-detection-project)
데이터 경로	/content/drive/MyDrive/pill_detection/

2. 탐색적 데이터 분석 (EDA)

2.1 데이터셋 기본 현황

AI Hub 경구약제 이미지 데이터를 가공한 데이터셋으로, Python(Pandas, OpenCV, Pillow)을 활용하여 COCO 포맷 JSON을 파싱하고 전체 현황을 파악했습니다.

항목	수치	비고
train 이미지 수	232장	test 대비 약 1/4
test 이미지 수	842장	평가용
JSON 파일 수	763개	이미지당 최대 4개 약 정보
고유 클래스(약) 수	56개	test는 40개
어노테이션 수	763개	
이미지 해상도	976 × 1280 px	100% 동일 — 리사이즈 불필요
색상 모드	RGB	
이미지-JSON 매칭	232개 (100%)	누락 없음

2.2 촬영 환경 분석

전체 데이터의 배경색이 100% 연회색, 조명이 100% 주백색으로 환경 다양성이 전혀 없습니다. 이는 실제 서비스 환경(다양한 배경, 자연광)에서 성능이 저하될 수 있는 핵심 위험 요소로, EDA 단계에서 사전에 파악하여 증강 전략에 반영했습니다.

항목	분포	위험도	대응 전략
배경색	연회색 100%	높음	Color Jitter, 배경 증강
조명색	주백색 100%	높음	밝기/채도 변조
촬영 방향	앞면 98.4% / 뒷면 1.6%	높음	뒷면 데이터 추가 수집 필요
촬영 각도	70도 / 75도 / 90도 균등	낮음	Rotate 증강으로 대응

2.3 클래스 불균형 분석

전체 56개 클래스 중 최다 클래스(일양하이트린정 2mg)는 153개, 최소 클래스는 3개로 최대 51배의 불균형이 존재합니다. Matplotlib을 활용한 바차트 시각화로 불균형 패턴을 파악했습니다.

항목	수치
전체 클래스 수	56개
최다 클래스	일양하이트린정 2mg — 153개
최소 클래스	다수 약물 — 3개
최대 불균형 비율	51배

소수 클래스 수 (3개 이하)	17개 (전체의 30%)
이미지당 알약 수	2개(7장) / 3개(151장) / 4개(74장)

2.4 메타데이터 분석

JSON 파일 내 40여 개 메타데이터 필드의 채움률을 분석하여 실제 서비스(UI) 연동 가능한 필드를 선별했습니다.

필드	설명	채움률	UI 활용
dl_name	제품명	100%	
dl_company	제조사	100%	
di_etc_otc_code	전문/일반 구분	100%	
dl_material	성분명	98.8%	
color_class1	색상	97.6%	
drug_shape	알약 모양	97.6%	
print_front	앞면 각인	95.5%	
print_back	뒷면 각인	64.1%	
line_front	앞면 분할선	25.3%	

2.5 주요 시각화 결과

Matplotlib을 활용하여 다음 항목을 시각화했습니다. 상세 시각화 결과는 EDA/eda_combined_최종.ipynb를 참조하세요.

시각화 항목	도구	주요 발견
클래스별 어노테이션 수 바차트	Matplotlib	최대 51배 불균형 확인
이미지당 알약 수 분포	Matplotlib	3개 65%, 4개 32%
촬영 환경 파이차트	Matplotlib	배경/조명 100% 단일
bbox 크기 분포 히스토그램	Matplotlib	평균 너비 250px, 높이 265px
각도별 bbox 위치 비교	Pandas + Matplotlib	70/75/90도 위치 크게 다름
메타데이터 채움률 바차트	Matplotlib	UI 연동 가능 필드 12개 선별
bbox 오버레이 시각화	OpenCV + Pillow	바운딩박스 정상 확인

2.6 전처리 및 증강 전략

EDA 결과를 토대로 Albumentations 라이브러리를 활용한 증강 파이프라인을 구성했습니다. bbox_params에 min_visibility=0.3을 설정하여 증강 후 잘린 바운딩 박스를 필터링했습니다.

단계	기법	파라미터	목적
①	LongestMaxSize + PadIfNeeded	max_size=1024	고해상도 유지

②	ColorJitter	p=0.7	조명/색상 환경 편향 해소
③	GaussNoise / MotionBlur	std=(0.01~0.05), p=0.4	노이즈 강건성
④	CoarseDropout	p=0.4	반쪽 알약 시뮬레이션
⑤	HFlip / VFlip / Rotate	p=0.5/0.3/0.5, limit=30	기하학적 다양성
⑥	Sharpen	p=0.3	각인 선명화
⑦	Normalize + ToTensorV2	ImageNet mean/std	정규화

※ GaussNoise std_range는 0~1 정규화 범위로 설정 (픽셀값 아님)

3. 모델 실험 과정

3.1 Round 1 — 아키텍처 탐색

PyTorch 기반 5가지 주요 객체 검출 아키텍처를 실험하여 초기 성능과 특성을 비교했습니다. 소규모 데이터셋(232장) 특성상 앵커 기반의 단순한 구조가 유리할 것으로 가정하고 실험을 진행했습니다.

모델	구조	특징	결과	비고
DETR	Transformer	전역 문맥 파악	낮음	소규모 데이터 부적합
Faster R-CNN (ResNet)	2-stage	높은 정확도	가장 높음	정확도 최우수
RetinaNet	1-stage	Focal Loss	중간	클래스 불균형 대응
SSD	1-stage	빠른 속도	중간	실시간 적합
YOLO	1-stage	속도+정확도	높음	균형 우수

→ Faster R-CNN 최고 성능, YOLO가 속도/정확도 균형에서 유리

3.2 Round 2 — 경량화 검토 및 방향 확정

실시간 서비스 요건을 고려하여 Faster R-CNN + MobileNetV3 백본으로 경량화를 시도했으나, mAP가 임계값 이하로 하락하여 Faster R-CNN 계열을 전면 제외했습니다. YOLO 계열을 메인 방향으로 최종 확정했습니다.

근거: YOLO는 1-stage detector로 추론 속도가 빠르고, Ultralytics 프레임워크를 통해 PyTorch 기반으로 손쉽게 파인튜닝이 가능합니다. 또한 TTA(Test Time Augmentation)와 앙상블 적용이 용이합니다.

3.3 Round 3 — YOLO 계열 세부 실험

Ultralytics 프레임워크를 활용하여 총 11개의 YOLO 변형 모델을 PyTorch로 학습했습니다. 모든 모델은 ImageNet 사전학습 가중치에서 파인튜닝했으며, AdamW 옵티마이저와 Cosine LR 스케줄러를 사용했습니다.

모델	파라미터	epoch	imgsz	mAP50-95	비고
YOLOv8s	11M	100	640	0.985	기준 모델
YOLOv8m	25M	100	640	0.990	
YOLOv11s	9M	100	640	0.989	최신 아키텍처
YOLOv11s_aug	9M	100	640	0.941	과증강 → 일반화 저하
YOLOv11m	20M	100	640	0.948	
YOLOv11l	25M	100	640	0.925	과적합 확인
RT-DETR-L	67M	100	640	0.020	소규모 데이터 부적합
YOLOv11s_1280	9M	150	1280	0.826	해상도↑ 오히려 역효과
YOLOv8m_1280	25M	72	1280	0.724	엘리스타핑
YOLOv8l	43M	100	640	0.991	신규 최고
YOLOv9m	20M	100	640	0.990	새 아키텍처

YOLOv11s_저lr	9M	100	640	0.992	lr=0.0005, 신규 최고
--------------	----	-----	-----	-------	------------------

3.4 실험을 통해 확인된 사실

발견	내용
모델 크기 ≠ 성능	v11l(25M)이 v11s(9M)보다 캐글 점수 낮음 → 소규모 데이터에서 과적합
과증강 역효과	augv2의 train mAP는 높지만 캐글 점수 낮음 → 일반화 성능 저하
해상도 불일치	1280px 학습 모델이 앙상블 추가 시 점수 하락 → 학습/테스트 불일치
RT-DETR 부적합	Transformer 계열은 소규모 데이터에서 CNN 계열보다 불리
저lr 효과	YOLOv11s에서 lr=0.0005로 낮추면 mAP 0.989 → 0.992 향상

4. 앙상블 전략

4.1 WBF vs Soft-NMS 비교

단일 모델보다 앙상블을 통해 예측의 다양성을 확보하면 최종 성능이 향상됩니다. ensemble-boxes 라이브러리를 활용하여 두 가지 방식을 비교 실험했습니다.

방식	메커니즘	Kaggle 점수	선택
WBF (Weighted Boxes Fusion)	검치는 박스들을 가중 평균으로 합쳐 하나의 박스 생성	0.97414 → 0.97581	채택
Soft-NMS	검치는 박스의 confidence 점수를 낮춰 자연스럽게 억제	0.96329	제외

4.2 WBF 파라미터 튜닝 과정

20회 이상의 Kaggle 제출을 통해 최적 파라미터를 탐색했습니다.

파라미터	탐색 범위	실험 결과	최적값
iou_thr	0.45 / 0.50 / 0.55 / 0.60	0.55에서 최고점	0.55
skip_box_thr	0.10 ~ 0.25	0.20 이하는 row 수 변화 없음 (한계점)	0.20
weights	균등 vs 다양한 조합	균등 [1.0, 1.0, 1.0] 최고	[1.0, 1.0, ...]
conf	0.15 / 0.25 / 0.30	변화 없음	0.25 (기본값)

4.3 앙상블 조합별 점수 변화 흐름

단계	조합	Kaggle 점수	변화
단일 모델	YOLOv8s 단독	0.95788	기준
2모델 앙상블	v8m + v11s	0.97077	+0.013
3모델 앙상블	v8s + v8m + v11s	0.97292	+0.002
파라미터 튜닝	3모델, iou=0.55, skip=0.20	0.97414	+0.001
v11l 추가 시도	4모델 + v11l	0.95524	0.019
augv2 추가 시도	4모델 + augv2	0.97077 미만	하락
1280 추가 시도	4모델 + 1280	0.91658	대폭 하락
Soft-NMS 시도	3모델 Soft-NMS	0.96329	하락
v8l 추가	4모델 + v8l	0.97581	+0.002 최고
6모델 시도	6모델 전체	0.97410	4모델보다 낮음

핵심 인사이트: 모델 수를 무조건 늘리는 것은 오히려 노이즈 증가로 성능을 저하시킵니다. 구조적으로 다양한 모델(파라미터 규모가 다른 모델)의 조합이 중요합니다.

5. 최종 결과

5.1 최종 앙상블 조합 비교

조합명	구성 모델	Kaggle 점수
기존 3모델	YOLOv8s + YOLOv8m + YOLOv11s	0.97414
4모델 + v9m	기존 3모델 + YOLOv9m	0.97374
4모델 + v11slr	기존 3모델 + YOLOv11s_저lr	0.97300
6모델 전체	기존 3모델 + v8l + v9m + v11slr	0.97410
4모델 + v8l (최종)	기존 3모델 + YOLOv8l	0.97581

5.2 최적 조합 확정

항목	내용
최종 모델 조합	YOLOv8s + YOLOv8m + YOLOv11s + YOLOv8l
앙상블 방식	WBF (Weighted Boxes Fusion)
iou_thr	0.55
skip_box_thr	0.20
weights	[1.0, 1.0, 1.0, 1.0] (균등)
TTA	각 모델 예측 시 augment=True 적용
최종 Kaggle 점수	0.97581 (mAP@[0.75:0.95])

6. 주요 의사결정 근거

의사결정	선택	근거
모델 방향 확정	YOLOv8	YOLO 계열 CNN 대비 속도/정확도 균형 우수. PyTorch 기반 Ultralytics 프레임워크로 TTA·양상블 적용 용이
학습 imgsz	640px	1024px는 4시간 내 3모델 동시 학습 불가. 640px로도 mAP 0.99 이상 달성
양상블 방식	WBF Soft-NMS	대비 0.97581 vs 0.96329. 박스를 합치는 방식이 이 데이터셋에 더 적합
TTA 적용	각 모델 예측 시	추론 다양성 확보. augment=True 한 줄로 적용 가능
v8i 선택	4번째 모델	파라미터 43M으로 기존 3모델(9~25M)과 구조적 다양성 확보. 양상블 다양성 극대화
모델 수 4개 제한	4모델 유지	6모델(0.97410) < 4모델(0.97581). 모델 수 증가가 노이즈 증가로 이어짐
균등 가중치	[1,1,1,1]	125가지 가중치 조합 탐색에서 균등이 최적. 특정 모델에 편향시키면 성능 저하

7. 한계점 및 개선 방향

7.1 과적합 위험

train 232장, 클래스당 평균 4~5장의 극소규모 데이터셋에서 mAP 0.97이 달성된 것은 모델이 데이터를 암기했을 가능성을 배제할 수 없습니다. 멘토 피드백에서도 K-Fold Cross Validation과 Grad-CAM 시각화를 통한 검증이 권고되었습니다.

7.2 도메인 편향 (OOD 취약성)

배경 100% 연회색, 조명 100% 주백색의 극단적으로 단일한 환경에서 학습된 모델은 실제 사용자 환경(다양한 배경, 자연광, 스마트폰 카메라)에서 성능이 급락할 수 있습니다. EDA 단계에서 이 문제를 사전에 파악했음에도 추가 데이터 수집의 한계로 해결하지 못했습니다.

7.3 향후 개선 방향

분류	개선 방향	기대 효과
데이터	다양한 배경/조명 환경 이미지 추가 수집	OOD 강건성 향상
데이터	뒷면 촬영 데이터 확보 (현재 1.6%)	뒷면 약 인식 가능
검증	K-Fold Cross Validation 적용	과적합 여부 정량적 확인
검증	Grad-CAM으로 학습 특징 시각화	모델이 약 외형을 보는지 확인
서비스	실제 스마트폰 촬영 이미지로 Blind Test	서비스 신뢰도 검증
경량화	Knowledge Distillation (v8l→v8s)	실시간 서비스 최적화
성능	더 다양한 하이퍼파라미터 탐색	추가 점수 향상 가능성

8. 결론

본 프로젝트에서는 232장의 소규모 경구약제 이미지 데이터셋으로 mAP@[0.75:0.95] 기준 0.97581을 달성했습니다.

EDA 단계에서 Pandas, OpenCV, Matplotlib을 활용하여 데이터의 환경 편향(배경/조명 100% 단일), 클래스 불균형(최대 51배), 각도별 위치 변화 등 핵심 문제를 사전에 파악했습니다. 이를 바탕으로 Albumentations 증강 파이프라인을 구성하여 모델의 강건성을 높였습니다.

PyTorch 기반 Ultralytics 프레임워크를 활용하여 총 12개 모델을 실험했습니다. 5가지 아키텍처 비교 → YOLO 계열 확정 → 세부 변형 실험의 체계적인 접근으로 각 의사결정의 근거를 데이터로 검증했습니다.

최종적으로 YOLOv8s + YOLOv8m + YOLOv11s + YOLOv8l의 WBF 앙상블이 최고 성능을 보였으며, 단순히 모델 수를 늘리는 것보다 구조적으로 다양한 모델의 조합이 중요함을 확인했습니다.

한계점: 소규모 데이터셋의 과적합 위험과 단일 환경 학습으로 인한 OOD 취약성이 존재합니다. 실제 서비스 적용 전 다양한 환경에서의 Blind Test와 K-Fold 검증이 필수적입니다.

최종 결과 요약	
최고 Kaggle 점수	0.97581 (mAP@[0.75:0.95])
최적 앙상블	YOLOv8s + YOLOv8m + YOLOv11s + YOLOv8l (WBF)
WBF 파라미터	iou_thr=0.55, skip_box_thr=0.20, weights=[1,1,1,1], TTA=True
총 실험 모델 수	12개 단일 모델 + 다수 앙상블 조합
총 캐글 제출 횟수	20회+
개발 환경	Google Colab Pro (T4 GPU), PyTorch, Ultralytics 8.4.46